



## 北京开源芯片研究院实习生笔试题目

只读 Non-Blocking Cache 的设计、实现和验证

报告类型： 设计、验证报告

姓 名： 王豪

联系电话： 18215880952

联系邮箱： 3502918558@qq.com

# 目录

|                                |    |
|--------------------------------|----|
| 1. 项目运行说明                      | 4  |
| 1.1 项目结构                       | 4  |
| 1.2 测试程序运行方式                   | 4  |
| 2. 项目概述                        | 4  |
| 2.1 整体项目概述                     | 4  |
| 2.2 Cache 设计概要                 | 4  |
| 2.3 验证方式概要                     | 5  |
| 3. Non-Blocking Cache 架构设计     | 7  |
| 3.1 Non-Blocking Cache 的整体架构图  | 7  |
| 3.2 Non-Blocking Cache 的对外接口介绍 | 8  |
| 3.3 各模块的设计                     | 10 |
| 3.3.1. CacheData (数据存储)        | 10 |
| 3.3.2. CacheTag (标签存储)         | 12 |
| 3.3.3. CacheCtrl (缓存控制器)       | 14 |
| 3.3.4. MSHRModule (未命中状态处理寄存器) | 19 |
| 4. 验证框架设计                      | 27 |
| 4.1. 功能测试总体思路                  | 27 |
| 4.2. FakeCPU 的设计思路             | 28 |
| 4.3. FakeRAM 的设计思路             | 30 |
| 5. 仿真与验证                       | 33 |
| 5.1. 仿真验证参数                    | 33 |
| 5.2. 验证结果                      | 33 |
| 5.3. 测试结果分析                    | 34 |

|                           |    |
|---------------------------|----|
| 5.3.1 对于一条 MISS 的请求 ..... | 35 |
| 5.3.2 对于一条 HIT 的请求 .....  | 40 |

# 1. 项目运行说明

## 1.1 项目结构

详见./NonBlockingCache/README.md

## 1.2 测试程序运行方式

详见./NonBlockingCache/README.md

# 2. 项目概述

## 2.1 整体项目概述

本项目设计并实现了一个支持多缺失请求的非阻塞只读 Cache，采用 Chisel 硬件描述语言编写。Cache 规格为 16KB 容量、4 路组相联、行大小 64 字节，通过 MSHR 机制支持多个未完成缺失请求，采用树型 PLRU 替换算法。

验证环境基于 ChiselTest 框架构建，包含模拟 CPU 请求的 FakeCPU 和模拟内存的 FakeNonBlockingRAM，通过随机地址序列和延迟对功能正确性进行全面测试。

项目完成了从设计、实现到验证的全流程，实现了高性能非阻塞只读 Cache 系统。

## 2.2 Cache 设计概要

Cache 规格为：容量 16KB，采用 4 路组相联结构，采用树型 PLRU（伪最近最少使用）替换算法，Cache 行大小为 64 字节。

其核心特点是支持请求合并与乱序响应：当发生 Cache 缺失（miss）时，系统不会阻塞后续的 CPU 请求，而是通过 MSHR（Miss Status Holding Register）的模块来追踪和管理最多 4 个未完成的、地址不重复的内存请求，以及 8 个可能地址部分重合的 Load 请求。

内存数据以突发传输模式返回（8 个周期，每次 64 位），在 Cache 内部被组装成 512 位的完整缓存行（Cache Line）。

整个设计包含标签存储（CacheTag）、数据存储（CacheData）、MSHR 控制（MSHRModule）和控制器（CacheCtrl）等模块，协同工作以实现对 CPU 读请求的高效、低延迟响应。

## 2.3 验证方式概要

该项目构建了一个完整、可重复的仿真测试环境，用于验证非阻塞 Cache 的功能正确性。验证环境主要利用了 ChiselTest（基于 ScalaTest）工具。FakeCPU 和 FakeRAM 则是利用该框架的 API 编写的特定测试激励组件。整个流程是在软件层面对 Chisel 生成的硬件模型进行周期精确的仿真。

其核心是通过两个模块化的测试组件来模拟 Cache 的上下游行为：

FakeCPU 模拟了 CPU 端的读写请求，它会按照固定的随机种子生成一系列具有局部性特征的地址序列，并按随机间隔发送请求，同时实时比对 Cache 返回的数据与预期值以验证 Cache 的行为正确性；

```
TestData > Req.txt
1  ===== CPU INIT REQUESTS =====
2  This data is generated with a fixed random seed during simulation testing.
3  The request data will contain a certain proportion of test hits/replacement cache samples, r.
4  For detailed code, see: src/test/scala/FakeCPU.scala
5  REQ00 : addr=0x8a505179bda1 rd=21
6  REQ01 : addr=0x9d83f5c229d5 rd=18
7  REQ02 : addr=0xef664a778da7 rd=11
8  REQ03 : addr=0x8186e5d77daa rd=27
9  REQ04 : addr=0x9f997f7fef7c rd=5
10 REQ05 : addr=0x93e560e23455 rd=20
```

由 FakeCPU 生成的具有局部性特征请求序列

FakeNonBlockingRAM 则模拟了下层内存，它会以随机的延迟接受 Cache 的请求，并以突发传输的方式（分 8 个周期返回一个完整的 512 位 Cache 行）返回预先生成的内存数据。

```
TestData > Ram.txt
1  Generated with a random seed during test program execution
2  For detailed code, see: src/test/scala/Tool.scala
3
4  SYSTEM RAM CONTENTS (512-bit)
5
6  Addr  Data
7
8  0x00  984bc005fc0d8e66
9        5bb2068bbe3e36d8
10       8858bde377b2b60c
11       f274dd02370d0be7
```

由 FakeNonBlockingRAM 随机生成的模拟内存数据

整个测试由 src/test/scala/CacheSpec.scala 进行调度，预设总的测试周期数和

请求数量，并驱动仿真逐步执行，直至所有测试用例完成或达到最大周期数后自动停止。该环境通过固定的随机种子确保了测试的可重复性，并通过打印详尽的地址、数据与周期信息，实现了对 Cache 命中、缺失、合并请求及乱序响应等关键行为的透明化验证。

```
[CPU ] RECIEVE [Time 23] REQ [ 1] rd=18 data=0xbc04a1f23d3505c0

[VERIFY] SendReq: 1
+-----+
| Addr      : 0x0009d83f5c229d5 |
| LineIndex : 0x27              |
| BankIndex : 0x02              |
+-----+
| Expected  : Data: 0xbc04a1f23d3505c0 RdIdx: 18 |
| Actual    : Data: 0xbc04a1f23d3505c0 RdIdx: 18 |
+-----+

-----> PASS

[CPU ] RECIEVE [Time 32] REQ [ 0] rd=21 data=0x3bc67b4522d425d6

[VERIFY] SendReq: 0
+-----+
| Addr      : 0x0008a505179bda1 |
| LineIndex : 0x36              |
| BankIndex : 0x04              |
+-----+
| Expected  : Data: 0x3bc67b4522d425d6 RdIdx: 21 |
| Actual    : Data: 0x3bc67b4522d425d6 RdIdx: 21 |
+-----+

-----> PASS

Tested Num: 2      All Test Num: 2
[info] CacheSpec:
[info] Non-BlockingCache
[info] - should using random delays through randomly generated test cases
[info] Run completed in 19 seconds, 787 milliseconds.
[info] Total number of tests run: 1
[info] Suites: completed 1, aborted 0
[info] Tests: succeeded 1, failed 0, canceled 0, ignored 0, pending 0
[info] All tests passed.
[success] Total time: 24 s, completed Feb 20, 2026, 12:33:03 PM
```

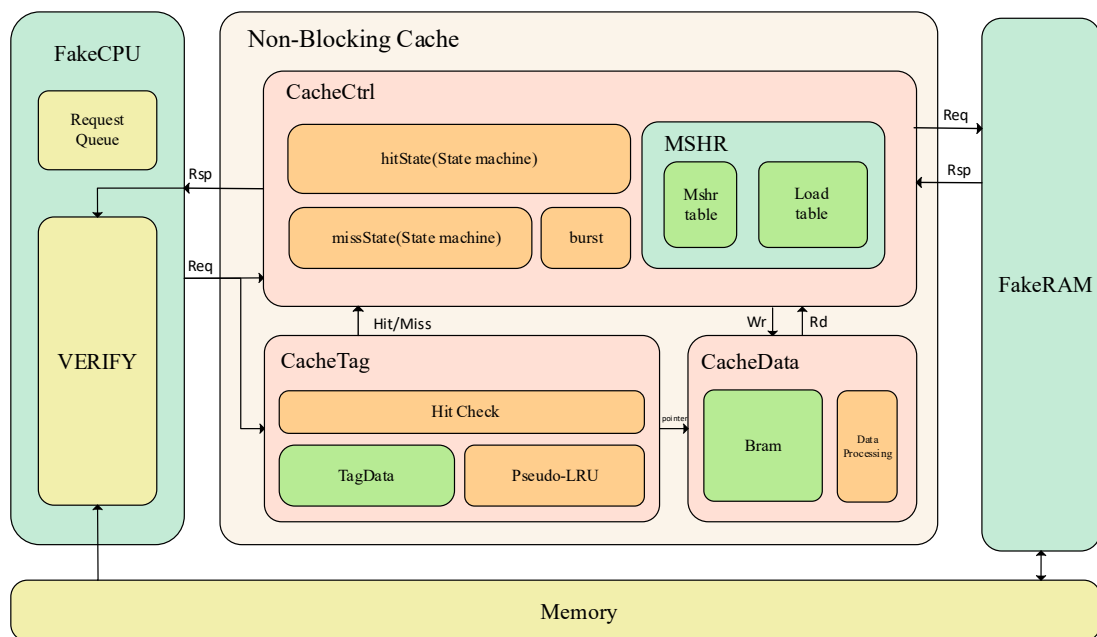
验证成功示例（2 项请求）

### 3. Non-Blocking Cache 架构设计

#### 3.1 Non-Blocking Cache 的整体架构图

Non-Blocking Cache 采用经典的模块化设计，包括：

- **CacheCtrl（缓存控制器）**：接收 CPU 请求，根据命中/未命中结果调度数据流，并管理 MSHR 与内存及 CPU 的交互。
- **CacheTag（标签存储）**：存储并比较地址标签，判断请求命中与否，并执行树型 PLRU 替换算法决定替换目标。
- **CacheData（数据存储）**：存储实际的缓存行数据，并根据地址偏移返回 CPU 所需的 64 位数据。
- **MSHRModule（未命中状态处理寄存器）**：核心的非阻塞控制模块，负责追踪、合并和管理多个未完成的缺失请求及其关联的 Load 请求。



## 3.2 Non-Blocking Cache 的对外接口介绍

### CPU-Cache 端:

此组接口用于接收 CPU 的读请求并返回数据，采用请求-地址有效与数据握手分离的类 SRAM 协议。

请求输入信号:

**req(输入, 1 位):** CPU 读请求有效信号，高电平表示当前周期 CPU 发起了有效的读请求。

**addr(输入, 48 位):** CPU 发出的物理读地址，位宽与 RISC-V RV64 架构的 48 位物理地址宽度匹配。

**rd\_idx\_in(输入, 5 位):** CPU 请求中携带的目标寄存器索引，Cache 将在返回数据时原样传回，用于 CPU 内部将数据写回正确的寄存器。

**sendReq\_in(输入, 16 位):** 由 CPU 生成的唯一请求标识号。Cache 会在后续的数据握手(data\_ok)响应中，通过 sendReq\_out 信号将同一个标识号返回，以支持乱序返回和请求匹配。

控制与响应输出信号:

**addr\_ok(输出, 1 位):** 地址握手成功信号。当 Cache 在本周期成功接收了 CPU 的 req 和 addr 时置高, 表示请求已被接受。无论是命中还是缺失, 只要资源允许, Cache 都应尽快回复此信号。

**data\_ok(输出, 1 位):** 数据握手成功信号。当请求的数据（无论是来自 Cache 命中还是 MSHR 返回）已就绪并出现在 rdata 上时置高, 且仅维持一个周期。

**rdata(输出, 64 位):** 返回给 CPU 的 64 位读数据。仅在 data\_ok 有效的周期, 其上的数据才有效。



**rd\_idx\_out(输出, 5 位):** 与 data\_ok 和 rdata 同时输出的目标寄存器索引, 其值等于对应请求的 rd\_idx\_in。

**sendReq\_out(输出, 16 位):** 与 data\_ok 和 rdata 同时输出的请求标识号, 其值等于对应请求的 sendReq\_in。CPU 可根据此标识将数据分发到正确的处理上下文。

#### **RAM-Cache 端:**

此组接口用于在 Cache 未命中时向下一级内存（如主存）发起读请求, 并接收返回的数据。接口支持请求接受握手和带突发传输的数据返回。

请求输出信号:

**mem\_req(输出, 1 位):** 内存读请求有效信号, 高电平表示 Cache 向内存发起了一次读请求。

**mem\_addr(输出, 48 位):** 发往内存的读地址。

**mem\_sendReq\_out(输出, 16 位):** 发往内存的请求标识号, 用于在内存乱序返回数据时进行匹配。

请求响应输入信号:

**mem\_addr\_ok(输入, 1 位):** 内存地址接受信号。当内存子系统在本周期成功接受了 mem\_req 和 mem\_addr 时置高。此信号为高后, Cache 需撤销此次请求避免重复。

数据返回输入信号:

**mem\_data\_valid(输入, 1 位):** 内存返回数据有效信号。在突发传输的每一个周期, 当 mem\_rdata 上的 64 位数据有效时, 此信号置高。

**mem\_data\_ok(输入, 1 位):** 内存突发传输完成信号。在突发传输的最后一个周期, 当最后一个 64 位数据有效 (即 mem\_data\_valid 也有效) 时, 此信号置高, 表示整个 Cache 行 (512 位) 数据传输完毕。

**mem\_sendReq\_back(输入, 16 位):** 内存返回数据所对应的请求标识号, 与之前 mem\_sendReq\_out 发出的标识匹配。

**mem\_rdata(输入, 64 位):** 内存返回的 64 位数据。在突发传输中, 每个 mem\_data\_valid 有效的周期传输 Cache 行的一部分, 共 8 个周期完成 512 位传输。

### 3.3 各模块的设计

#### 3.3.1. CacheData (数据存储)

##### [1] 模块功能概述

- **功能:** 缓存层次结构中的数据存储部分。
- **关键描述:** 数据存储: 为 4 个并行的存储体 (Way) 提供物理存储单元。数据写入: 当发生缓存缺失并从内存获取到新数据后, 将完整的 512 位缓存行写入由替换算法指定的 Way 中。数据读取: 当 CPU 请求命中缓存时, 根据命中的 Way 和请求地址中的偏移量, 从对应的 512 位缓存行中选取正确的 64 位数据返回给 CPU。

##### [2] 模块详情描述

###### a) 计算回填、请求信息

```
39      /*-----计算回填信息-----*/
40      val refill_offset = io.refillAddr_fromCtrl(OFFSET_MSB , OFFSET_LSB)
41      val refill_index  = io.refillAddr_fromCtrl(INDEX_MSB  , INDEX_LSB)
42      val refill_tag    = io.refillAddr_fromCtrl(TAG_MSB    , TAG_LSB)
43
44      /*-----计算请求信息-----*/
45      val req_offset    = io.reqAddr_fromCtrl(OFFSET_MSB    , OFFSET_LSB)
46      val req_index     = io.reqAddr_fromCtrl(INDEX_MSB     , INDEX_LSB)
47      val req_tag       = io.reqAddr_fromCtrl(TAG_MSB       , TAG_LSB)
48      //从cache块中选择数据 【 512bits 选 64bits 】
```

该模块接收来此外层的地址信息并做计算。

## b) 实例化存储器信息

```
51  /*-----开始制作存储器-----*/
52
53  /*第一路-cacheWay0*/
54  val Way0_brams = Seq.fill(CACHE_LINE)(Module(new CacheDualPortBRAM))
55  val rdata_way0 = Wire(Vec(CACHE_LINE, UInt(64.W)))
56
57  for (n <- 0 until CACHE_LINE) {
58    /*--写信号--*/
59    val wr_en    = io.replace_fromCtrl && io.pointer_fromTag(0)
60    val wr_index = refill_index
61    val startBit = n * 64
62    val endBit   = (n + 1) * 64 - 1
63    val wr_data  = io.newCacheline_fromCtrl(endBit, startBit)
64    Way0_brams(n).io.wr_en    := wr_en
65    Way0_brams(n).io.wr_index := wr_index
66    Way0_brams(n).io.wr_data  := wr_data
67
68    /*--读信号--*/ /*单周期读出*/
69    val rd_en    = io.req && bank_sel(n) && io.hit_fromTag(0)
70    val rd_index = req_index
71    val rd_data  = Way0_brams(n).io.rd_data
72    Way0_brams(n).io.rd_en    := rd_en
73    Way0_brams(n).io.rd_index := rd_index
74    rdata_way0(n)            := rd_data
75  }
```

将存储器信息进行实例化，并正确将 Bram 各模块的信息连接上。

## c) 数据处理

```
153  val selected_way0_data = Wire(UInt(64.W))
154  val selected_way1_data = Wire(UInt(64.W))
155  val selected_way2_data = Wire(UInt(64.W))
156  val selected_way3_data = Wire(UInt(64.W))
157
158  selected_way0_data := Mux1H(bank_sel.asBools, rdata_way0)
159  selected_way1_data := Mux1H(bank_sel.asBools, rdata_way1)
160  selected_way2_data := Mux1H(bank_sel.asBools, rdata_way2)
161  selected_way3_data := Mux1H(bank_sel.asBools, rdata_way3)
162
163  val hit_selected_data = Wire(UInt(64.W))
164
165  when(io.hit_fromTag(0)) {
166    hit_selected_data := selected_way0_data
167  }.elsewhen(io.hit_fromTag(1)) {
168    hit_selected_data := selected_way1_data
169  }.elsewhen(io.hit_fromTag(2)) {
170    hit_selected_data := selected_way2_data
171  }.elsewhen(io.hit_fromTag(3)) {
172    hit_selected_data := selected_way3_data
173  }.otherwise {
174    hit_selected_data := 0.U // 没有命中时输出0
175  }
176
177  io.rdata_toCtrl := hit_selected_data
```

由于一个 Cache 行大小是 512bits，当访问 Cache 行的时候，需要根据地址的 offset 信息在 512bits 中选择正确的 64bits 向外传递。

### 3.3.2. CacheTag（标签存储）

#### [1] 模块功能概述

缓存标签（CacheTag）模块是组相联缓存系统的核心控制部件，负责将 CPU 的访存地址映射到具体的缓存行，并判断访问是否命中。本模块具有以下核心功能：

1. **地址解析与索引：**将输入的 48 位物理地址解析为标签(Tag)、索引(Index)和块内偏移(Offset)三个字段，其中索引用于在 4 个并行的存储体(Way)中定位到具体的缓存组 (Set)。
2. **标签存储与并行匹配：**为每个 Way 维护一个标签存储器（包含有效位和标签位）。在收到请求时，模块并行地将请求标签与索引所对应的 4 个 Way 中存储的标签进行比较，生成 4 位的命中向量（独热码）。
3. **缺失判定与替换指针生成：**根据命中向量的结果，即时判定本次访问是命中 (Hit) 还是缺失 (Miss)。当发生缺失且需要分配新缓存行时，模块运行 Tree-PLRU（伪最近最少使用）替换算法，计算并输出一个 4 位的独热码（替换指针），指示应被替换的 Way。
4. **替换算法状态维护：**无论是命中访问还是新缓存行回填，都会更新对应组的 LRU 状态，确保算法能近似追踪各 Way 中被访问的“新旧”顺序。
5. **标签更新：**当缓存控制器完成缺失处理并回填新数据时，本模块根据“替换指针”和回填地址，将新的标签（及有效位）写入对应 Way 的标签存储器中，完成缓存目录的更新。

## [2] 模块详情描述

### a) 命中/缺失判断

```
57 val validTag = Cat(1.U(1.W), req_tag)
58 //1.Way0
59 val tag_way0 = RegInit(
60   VecInit(Seq.fill(INDEX_NUM)(
61     0.U((TAG_WD + 1).W) /*1位的Valid + TAG_WD位的Tag*/
62   ))
63 )
64 > val hit_way0 = ...
69 //2.Way1
70 > val tag_way1 = RegInit( ...
74 )
75 > val hit_way1 = ...
79 //3.Way2
80 > val tag_way2 = RegInit( ...
84 )
85 > val hit_way2 = ...
90 //4.Way3
91 > val tag_way3 = RegInit( ...
95 )
96 > val hit_way3 = ...
100 val hit = Cat(hit_way3, hit_way2, hit_way1, hit_way0) /*命中信号*/
101 val miss = req && ~(hit_way0 || hit_way1 || hit_way2 || hit_way3); /*缺失信号*/
```

为 4 路（Way0 至 Way3）组相联缓存的标签存储器实例化并初始化，每个标签由 1 位有效位和实际的标签位组成。当请求有效时，系统会并行地将拼接了有效位的当前请求标签与每个缓存路中由索引指定的存储标签进行比对，生成 4 个独立的命中信号。这些命中信号随后被组合成一个 4 位向量输出，以指示具体命中的缓存路（采用独热编码），同时通过逻辑运算判断是否发生缺失（即所有缓存路均未命中）。最终的命中与缺失信号被输出至模块接口，为上层缓存控制器提供决策依据。

### b) 缓存标签存储器的更新

```
108 val replace_offset = io.replace_addr(OFFSET_MSB, OFFSET_LSB)
109 val replace_index = io.replace_addr(INDEX_MSB, INDEX_LSB)
110 val replace_tag = io.replace_addr(TAG_MSB, TAG_LSB)
111 //当【新替换数据到来】且【替换指针指向this Way】
112 when (replace_cache && replacePointer(0)) {
113   tag_way0(replace_index) := Cat(1.U(1.W), replace_tag)
114 }
115 //当【新替换数据到来】且【替换指针指向this Way】
116 when (replace_cache && replacePointer(1)) {
117   tag_way1(replace_index) := Cat(1.U(1.W), replace_tag)
118 }
119 //当【新替换数据到来】且【替换指针指向this Way】
120 when (replace_cache && replacePointer(2)) {
121   tag_way2(replace_index) := Cat(1.U(1.W), replace_tag)
122 }
123 //当【新替换数据到来】且【替换指针指向this Way】
124 when (replace_cache && replacePointer(3)) {
125   tag_way3(replace_index) := Cat(1.U(1.W), replace_tag)
126 }
```

当上层控制器发出替换信号（replace\_cache 有效）时，代码会根据输入的替

换地址 (replace\_addr) 解析出目标索引 (replace\_index) 和标签 (replace\_tag)。同时, 一个由替换算法(如 PLRU)计算出的 4 位独热码替换指针(replacePointer)指示了具体需要更新的缓存路 (Way)。通过四个独立的条件判断语句 (when), 分别检查替换指针的每一位。当某一位为高且替换信号有效时, 便将该路标签存储器中由 replace\_index 指定的条目更新为包含 “有效位” 和 replace\_tag 的新标签值, 从而完成缓存目录的刷新, 为后续的命中判断做好准备。

### c) 基于二叉树的伪最近最少使用 (Tree-PLRU) 替换算法

```

165 when (hit_way0 || (replace_cache && replacePointer(0))) {
166   lru(use_index) := Cat( //这里的地址待完善
167     1.U(1.W), // b2
168     1.U(1.W), // b1
169     lru(use_index)(0) // b0 不变
170   )
171 }
172 > .elsewhen (hit_way1 || (replace_cache && replacePointer(1))) { ...
173 }
174 > .elsewhen (hit_way2 || (replace_cache && replacePointer(2))) { ...
175 }
176 > .elsewhen (hit_way3 || (replace_cache && replacePointer(3))) { ...
177 }
178 }
179 val b2 = lru(replace_index)(2)
180 val b1 = lru(replace_index)(1)
181 val b0 = lru(replace_index)(0)
182 replacePointer :=
183   Mux(!b2,
184     Mux(!b1, 1.U, 2.U), /*0001:way0 0010:way1*/
185     Mux(!b0, 4.U, 8.U) /*0100:way2 1000:way3*/
186   )

```

具体而言, 该算法为缓存中的每个组 (Set) 维护一个 3 位的 LRU 状态机 (b2, b1, b0), 其结构模拟了一棵二叉树, 叶子节点对应 4 个缓存路 (Way)。

每当某个缓存路被访问 (无论是命中还是因缺失而回填新数据), 算法就会沿着从根节点到该叶子节点的路径, 将经过的节点位 (b2, b1, b0) 翻转, 使其指向另一个子树, 从而标记该路为 “最近使用过”。

当需要选择替换目标时, 算法从根节点 b2 开始, 根据其当前值 (0 或 1) 选择左或右子树, 再根据下一级节点 (b1 或 b0) 的值最终确定一个叶子节点 (即一个 Way), 其对应的 4 位独热编码即为输出的 replacePointer。这个机制以近似的方式追踪了各缓存路的使用顺序, 并以较低的硬件开销实现了高效的替换决策。

### 3.3.3. CacheCtrl (缓存控制器)

## [1] 模块功能概述

缓存控制器（CacheCtrl）模块是非阻塞缓存系统的核心调度中枢，负责协调 CPU 请求、MSHR（未命中状态处理寄存器）、内存接口以及数据存储阵列之间的所有交互。本模块具有以下核心功能：

1. **状态机调度：**维护独立的状态机分别处理缓存命中请求与缓存缺失请求，确保两者可并行处理，提高效率。
2. **命中路径响应：**当请求命中缓存时，从 CacheData 数据存储阵列读取数据，并将正确的 64 位数据连同其对应的请求标识一并返回给 CPU。
3. **缺失路径处理：**当请求未命中时，将请求及其元数据转发给 MSHR 模块，并即时接收 MSHR 处理完成后的数据返回给 CPU。
4. **内存接口桥接：**将 MSHR 发出的内存读请求转发给下层内存，并处理内存的突发传输响应，将多个 64 位数据组装成 512 位的完整缓存行后提交给 MSHR。
5. **更新 CacheData 控制：**接收来自 MSHR 的缓存行回填请求，将新数据及其地址转发给数据存储和标签存储模块，以完成缓存的更新。

## [2] 模块详情描述

### a) 处理缓存命中响应（CPU 端、MSHR 端）的仲裁与数据选择状态机

```
71 | /*=====
72 | 1. 处理Hit的状态机
73 | 处理：对于来自CPU的Hit请求 以及 来自MSHR的已准备好的表项请求
74 | 这俩处理需要在同一状态机，因为他们可能同时发生
75 | =====*/
76 |
77 | val hitState = RegInit(IDLE)
78 | switch(hitState) {
79 |
80 > | is(IDLE) { ...
90 | | }
91 > | is(MSHR_OK) { ...
93 | | }
94 > | is(CPU_BACK_ADDROK) { ...
96 | | }
97 > | is(CPU_BACK_DATA) { ...
99 | | }
100 | }
101 | io.reqAddr_toData := io.req_addr_fromCPU
102 | val rdata_fromData_reg = RegNext( io.rdata_fromData )
103 | val rdIdx_fromCPU_reg = RegNext( io.rdIdx_fromCPU )
104 | val reqNum_fromCPU_reg = RegNext( io.reqNum_fromCPU )
105 |
106 | val dataOK_fromCache = ( hitState === CPU_BACK_DATA )
107 | val dataOK_fromMSHR = ( hitState === MSHR_OK )
108 | mshr.io.cpu_rsp_ready := ( hitState === MSHR_OK )
109 |
110 > | io.rdIdx_toCPU := Mux(dataOK_fromCache, ...
115 > | io.reqNum_toCPU := Mux(dataOK_fromCache, ...
120 > | io.rdata_toCPU := Mux(dataOK_fromCache, ...
126 | io.data_ok_toCPU := dataOK_fromCache || dataOK_fromMSHR
```

具体而言，该状态机负责处理并调度两种不同的命中数据流：一是来自缓存数据存储的直接命中请求，二是来自 MSHR 模块的、针对先前缺失请求现已准备好的数据。

状态机定义了四个状态：IDLE、MSHR\_OK、CPU\_BACK\_ADDROK 和 CPU\_BACK\_DATA。在 IDLE 状态下，它根据优先级检测上述两种请求，并进入相应的处理路径。

对于直接命中请求，状态机经历 CPU\_BACK\_ADDROK 和 CPU\_BACK\_DATA 两个状态，以寄存从 CacheData 读取的数据和请求元数据；

对于 MSHR 返回的数据，则直接进入 MSHR\_OK 状态。

最终，基于当前所处的状态（dataOK\_fromCache 或 dataOK\_fromMSHR），代码通过多路选择器（Mux）从正确的源（缓存的寄存器或 MSHR 的输出）选取对应的数据、寄存器索引和请求号，并驱动输出到 CPU 接口，同时生成数据有效信号（data\_ok\_toCPU），从而确保 CPU 能在正确的时间点收到准确的数据响应。



## b) 处理缓存缺失并分配 MSHR 表项的简单状态机

```

129 | /*=====
130 | 2. 处理Miss的状态机
131 | 包括：对新进来的Miss请求进行写MSHR操作
132 | =====*/
133 |
134 | val missState = RegInit(IDLE)
135 | switch(missState) {
136 > | is(IDLE) { ...
142 | }
143 > | is(WRITE_MSHR) { ...
145 | }
146 |
147 | }
148 |
149 | mshr.io.cpu_req_valid := (missState === WRITE_MSHR)
150 | mshr.io.cpu_req_addr := io.req_addr_fromCPU
151 | mshr.io.cpu_req_id   := io.reqNum_fromCPU
152 | mshr.io.cpu_req_idx  := io.rdIdx_fromCPU
153 |
154 | io.addr_ok_toCPU := ( hitState === CPU_BACK_ADDROK ) || //Hit时
155 |                  ( missState === WRITE_MSHR )         //Miss时

```

具体而言，该状态机用于处理来自 CPU 的缓存缺失请求。当系统处于 IDLE 状态时，如果同时满足三个条件——CPU 请求有效、标签模块报告缺失、且 MSHR 模块有空闲条目可接受新请求，状态机便会进入 WRITE\_MSHR 状态。

在此状态下，代码将当前的缺失请求地址、请求标识号和寄存器索引驱动到 MSHR 模块的对应接口，完成新缺失请求的登记。状态在 WRITE\_MSHR 只停留一个周期，随后立即返回 IDLE，以准备接收下一个缺失请求。

同时，代码会综合命中处理状态和缺失处理状态，生成对 CPU 的地址确认信号（addr\_ok\_toCPU），无论本次请求是命中还是缺失，都向 CPU 反馈请求已被成功接收，以达到非阻塞的效果。

## c) MSHR 模块向内存请求接口的直连接通

```

157 | /*=====
158 | 3. MSHR向内存的请求接口
159 | =====*/
160 |
161 | io.req_toMem      := mshr.io.mem_req_valid  //:= Output(Bool())           // 内存请求有效
162 | io.addr_toMem     := mshr.io.mem_req_addr   //:= Output(UInt(48.W))        // 内存请求地址
163 | io.reqNum_toMem    := mshr.io.mem_req_id     //:= Output(UInt(16.W))        // 内存请求ID
164 | mshr.io.mem_req_ready := io.addr_ok_fromMem //:= Input(Bool())             // 内存请求是否被接受

```

具体而言，它作为顶层控制器与底层 MSHR 模块之间的简单连接桥梁，将 MSHR 模块内部生成的、针对缺失缓存行的内存读请求信号（包括请求有效、地址和唯一标识号）直接透传给 Cache 的外部内存接口。

同时，它也负责将来自内存端的地址接收确认信号（`addr_ok_fromMem`）回传给 MSHR 模块，以告知 MSHR 其发出的内存请求是否已被成功接收。

#### d) 接收内存突发传输数据并组装缓存行的核心逻辑

```
166 | /*=====
167 | 4. 内存响应到MSHR的接口 主要处理突发传输
168 | =====*/
169 | mshr.io.mem_rsp_id      := RegNext(io.reqNum_back_fromMem) //唯一标识：配合突发传输需要延迟一个周期
170 > //Mem输入: ...
174 > //MSHR输入: ...
178 | // 突发传输
179 | val burst_counter = RegInit(0.U(3.W))
180 | val data_buffer = Reg(Vec(8, UInt(64.W)))
181 | val rsp_valid_reg = RegInit(false.B)
182 | // 当data_valid有效时，接收数据
183 > when(io.data_valid_fromMem) { ...
195 | }.otherwise {
196 |     rsp_valid_reg := false.B
197 |     for (i <- 0 until 8) {
198 |         data_buffer(i) := 0.U
199 |     }
200 | }
201 | mshr.io.mem_rsp_valid := rsp_valid_reg
202 > mshr.io.mem_rsp_data := Cat(...
```

具体而言，它处理来自内存接口的突发传输响应。每次当内存数据有效信号（`data_valid_fromMem`）为高时，代码将接收到的 64 位数据（`rdata_fromMem`）顺序存储到一个 8 个条目的向量缓冲区（`data_buffer`）中，该存储位置由一个 3 位计数器（`burst_counter`）指定。

在接收每个数据的同时，代码会检查突发传输结束信号（`data_ok_fromMem`）。如果该信号为高，表示当前是 8 个数据传输中的最后一个，计数器将被清零，并置位响应有效寄存器（`rsp_valid_reg`），标志着 512 位的完整缓存行已准备就绪；否则，计数器递增，继续等待后续数据。

最终，当 `rsp_valid_reg` 为高时，代码将缓冲区的 8 个 64 位数据逆序拼接成一个完整的 512 位缓存行，并连同延迟一个周期对齐的请求标识（`mem_rsp_id`）一起输出到 MSHR 模块，完成一次完整的缓存行数据交付。

### e) 从 MSHR 模块到缓存数据与标签存储的写回接口连接

```
207 | /*=====
208 | | 5. MSHR写CacheData和CacheTag的接口
209 | | =====*/
210 |
211 | io.replace_cache := mshr.io.cache_wr_valid
212 | io.refill_addr   := mshr.io.cache_wr_addr
213 | io.newCacheline := mshr.io.cache_wr_data
```

将 MSHR 模块生成的缓存行回填命令直接透传给缓存的数据和标签存储模块。当 MSHR 完成对某个缺失请求的处理并获取到完整的 512 位缓存行数据后，它会通过 `cache_wr_valid` 信号触发一次写操作，并通过 `cache_wr_addr` 和 `cache_wr_data` 信号分别提供要写入的缓存地址和完整的缓存行数据。

CacheCtrl 模块在这里仅作为连接桥梁，将这些信号直接驱动到对应的输出端口（`io.replace_cache`、`io.refill_addr`、`io.newCacheline`），从而将控制权移交给下游的 CacheData 和 CacheTag 模块，由它们完成实际的存储阵列更新。

### 3.3.4. MSHRModule（未命中状态处理寄存器）

#### [1] 模块功能概述

MSHR（Miss Status Holding Register）模块是非阻塞缓存系统中的核心控制部件，负责在缓存未命中时追踪和管理最多 4 个未完成的、地址不重合的内存请求，并关联最多 8 个与这些请求部分或完全重合的 Load 请求。本模块具有以下核心功能：

1. **未命中请求接纳与合并：**根据新请求地址是否与现有 MSHR 表项重合，决定将其分配给新表项或合并到现有请求，并为每个请求分配 Load 表项以追踪具体访存细节。
2. **内存请求调度：**实时查找并发出已就绪但尚未发送的未命中请求，将地址和唯一标识传递给内存接口，并在收到内存接受确认后标记请求为“已发送”。
3. **内存响应处理与数据广播：**根据返回数据包中的唯一标识，找到对应的 MSHR 表项存储完整缓存行，并将该行数据广播给所有关联的、正在等待数据的 Load 表项，为每个 Load 请求选择其所需的 64 位数据块。

4. **缓存更新控制**：在收到完整缓存行数据后，立即生成写 Cache 信号，将新数据及其地址传递至 CacheData 模块，完成缓存行的填充。
5. **CPU 响应调度**：查找已获得数据的 Load 请求，在 CPU 准备好接收时，将其数据和请求标识返回给 CPU，并标记该请求为“已完成”。

## [2] 模块详情描述

### a) 定义核心表项数据结构

```

49  /*=====
50  | 1. MSHR表项定义
51  =====*/
52  val MSHR_ENTRIES = 4
53  val MSHR_ID_WIDTH = 2 // 4个entry需要2位
54
55  // MSHR表项数据结构
56 > class MSHREntry extends Bundle { ...
64  }
65
66  // MSHR寄存器数组
67 > val mshr_entries = RegInit(VecInit(Seq.fill(MSHR_ENTRIES){ ...
77  }))
78
79  /*=====
80  | 2. Load表项定义
81  =====*/
82  val LOAD_ENTRIES = 8
83  val LOAD_ID_WIDTH = 3
84
85  // Load表项数据结构
86 > class LoadEntry extends Bundle { ...
95  }
96
97  // Load寄存器数组
98 > val load_entries = RegInit(VecInit(Seq.fill(LOAD_ENTRIES){ ...
109  })

```

MSHR 表项（用于追踪未完成内存请求）和 Load 表项（用于追踪挂起的 CPU 读请求）。

具体而言，它首先定义了 MSHR 表项的数据结构 MSHREntry，包含请求有效性、发出状态、数据接收状态、缓存写入状态、地址、数据和唯一 ID 等字段，并实例化了一个包含 4 个此类表项的寄存器数组 mshr\_entries，所有字段初始化为无效或零值。

接着，代码定义了 Load 表项的数据结构 LoadEntry，用于关联到具体的 MSHR 条目，记录 CPU 请求的元数据（如目标寄存器、块内偏移）以及数据返回状态，并实例化了一个包含 8 个此类表项的寄存器数组 load\_entries。这两组表项共同构成了 MSHR 模块内部跟踪和管理多个未完成缓存缺失请求及其关联 CPU 负载的状态存储基础。

## b) MSHR 模块内部状态信号的提取与状态标志计算

```
111  /*=====
112  | 3. 状态信号 做出状态信息向外传或者后面使用
113  =====*/
114  // MSHR状态
115  val mshr_valid_vec = mshr_entries.map(_.valid) //将mshr table的valid字段提取出来 该表项空不空
116  val mshr_issued_vec = mshr_entries.map(_.issued) //将mshr table的issued字段提取出来 该表项请求被发出去没有
117  val mshr_data_valid_vec = mshr_entries.map(_.data_valid) //将mshr table的data valid字段提取出来 该表项的数据有没有取到
118  val mshr_written_vec = mshr_entries.map(_.written_cache) //将mshr table的written cache字段提取出来 该表项的数据有没有被写入到c
119
120  val mshr_full = mshr_valid_vec.reduce(_ && _) // 对valid字段进行and操作 指示mshr table是否已经满了
121  val mshr_empty = !mshr_valid_vec.reduce(_ || _) // 对valid字段进行or操作并取反 指示mshr table是否空的【调试用】
122
123  // Load状态 - 与上面一样的操作
124  val load_valid_vec = load_entries.map(_.valid)
125  val load_data_valid_vec = load_entries.map(_.data_valid)
126  val load_returned_vec = load_entries.map(_.returned_cpu)
127
128  val load_full = load_valid_vec.reduce(_ && _)
129  val load_empty = !load_valid_vec.reduce(_ || _)
```

具体而言，该代码从已定义好的 MSHR 表项数组和 Load 表项数组中，分别提取出每个表项的关键状态字段（如有效位 `valid`、请求发出位 `issued`、数据有效位 `data_valid` 等），并组合成表示整个数组当前状态的向量（例如 `mshr_valid_vec`）。

接着，通过对这些状态向量进行逻辑的归约操作，计算出表征 MSHR 表整体是否已满（`mshr_full`）、是否为空（`mshr_empty`）以及 Load 表是否已满（`load_full`）、是否为空（`load_empty`）的全局状态信号。这些信号为模块后续的任务调度（如请求接收、内存访问、数据返回）提供了关键的决策依据，并可通过调试接口输出，便于监控模块运行状态。

## c) 任务 A：接收新的 miss 请求

```
141  // [1]. 对于从外面传进来的地址cpu_req_addr
142  // 需要检测这个地址 能否写入到MSHR 以及 如何写
143
144  // a. 检查地址是否与现有MSHR表项重合的重合情况 并且找出来
145  val addr_match = mshr_entries.zipWithIndex.map { case (entry, i) =>
146  | entry.valid && (entry.addr(47,6) === io.cpu_req_addr(47,6))
147  | }
148  val addr_match_vec = VecInit(addr_match)
149  val addr_match_any = addr_match_vec.reduce(_ || _) //是否匹配某一个
150  val match_mshr_id = PriorityEncoder(addr_match_vec) //编码找出是哪一个表项与现有的请求重合
151
152  // b. 新miss请求必然会写入load table中 只需检测该表项是否非满
153  val load_free_vec = load_entries.zipWithIndex.map { case (entry, i) =>
154  | !entry.valid || (entry.data_valid && entry.returned_cpu) // 找出空闲的表项或已完成的表项*****
155  | }
156  val load_free_any = load_free_vec.reduce(_ || _) // 指示load table是否有空闲
157
158  // c. 综合以上两table的条件, 指示能否接受新miss请求的条件
159  val can_accept_new = ( !mshr_full && !load_full ) || (addr_match_any && load_free_any) // 非空 或者 地:
160  io.cpu_can_accept := can_accept_new
```

首先检查新请求的地址（`io.cpu_req_addr` 的位[47:6]，即标签和索引部分）是否与任何一个现有有效的 MSHR 表项地址重合，生成一个匹配向量（`addr_match_vec`），并据此判断是否存在重合（`addr_match_any`）以及是哪一个

表项重合（`match_mshr_id`）。

同时，它检查 Load 表是否有空闲条目（`load_free_any`）。最终的准入决策（`can_accept_new`）遵循一个关键规则：只有当 MSHR 表和 Load 表都未滿，或者新请求地址与某个现有 MSHR 表项重合并且 Load 表有空闲时，新的缺失请求才能被接受。这个结果通过 `io.cpu_can_accept` 信号输出，告知上层控制器（CacheCtrl）当前是否可以发起一个新的缺失请求。

```
162 // [2]. 开始查找空闲表项的具体位置【找出写表项地址】
163
164 // a. 查找空闲的MSHR表项
165 val mshr_free_vec = mshr_entries.zipWithIndex.map { case (entry, i) =>
166   | !entry.valid || (entry.data_valid && entry.written_cache && entry.valid) // 空闲或已完成
167 }
168 val mshr_free_oh = PriorityEncoderOH(mshr_free_vec)
169 val mshr_free_id = OHToUInt(mshr_free_oh) // 找到那个空闲的mshr table位置
170 val mshr_free_any = mshr_free_vec.reduce(_ || _)
171
172 // 查找空闲的Load表项
173 val load_free_oh = PriorityEncoderOH(load_free_vec)
174 val load_free_id = OHToUInt(load_free_oh) // 找到那个空闲的load table位置
```

通过遍历 MSHR 条目数组，根据每个条目的状态（是否有效、数据是否已接收、是否已写回缓存）判断其是否空闲或已完成，生成一个布尔向量 `mshr_free_vec`。接着，通过优先级编码器 `PriorityEncoderOH` 找到该向量中第一个为真的位置，并将其转换为独热码 `mshr_free_oh`，再解码为二进制索引 `mshr_free_id`，同时计算是否存在任何空闲条目 `mshr_free_any`。对于 Load 表，它直接使用先前计算好的 `load_free_vec` 执行相同的优先级编码和解码流程，得到空闲 Load 条目的独热码索引 `load_free_oh` 和二进制索引 `load_free_id`。

```
187 // 更新MSHR表项
188 > when(write_mshr) { ...
189 }
190
191 // 更新Load表项
192 // 这里还有一点需要注意的，在只写load table的情况下（mshr table已经有了重合的请求）
193 // 这个时候就可能出现，这个新请求写入load table的时机 和 重合请求返回来的时机 是同一个周期
194 // 所以在这里需要进行一个对比判断是否有这种情况，如果有直接把返回的数据写入load table
195 // 不然新请求会被一直卡在那里，或者后面得到其他请求的错误的数据
196
197 val rsp_match_any = Wire(Bool())
198 val rsp_match_id = Wire(UInt(MSHR_ID_WIDTH.W))
199
200 // 判断是否会出现时机重合的情况
201 val match_rsp = io.mem_rsp_valid && rsp_match_any && (mshr_entries(rsp_match_id).addr(47,6) == io.cpu_req_addr(47,6))
202
203 val special_data_valid = Mux(match_rsp, true.B, false.B)
204
205 val rsp_selected_data = Wire(UInt(64.W))
206 > rsp_selected_data := Mux1H( ...
207 )
208 val special_data = Mux(match_rsp, rsp_selected_data, 0.U)
209
210 > when(write_load) { ...
211 }
212
```

根据条件（请求有效、可被接受、地址匹配情况、表项空闲情况）生成了写 MSHR 表信号 `write_mshr` 和写 Load 表信号 `write_load`。



当 `write_mshr` 有效时,会在一个空闲的 MSHR 表项中登记新的未命中请求,初始化其所有状态。

当 `write_load` 有效时,会在一个空闲的 Load 表项中登记新的 CPU 请求,并将其与一个 MSHR 表项(新分配的或已存在的)关联。

此逻辑的关键优化在于处理一种特殊情况:当新请求的地址与一个已发出的 MSHR 请求重合,并且恰好在同一周期该 MSHR 请求的内存数据返回。代码通过 `match_rsp` 逻辑检测到此时序重合,并直接从当前返回的 512 位内存数据中选出该新请求所需的 64 位数据,立即写入对应的 Load 表项,并将其 `data_valid` 置为真,从而避免了该请求被不必要地阻塞,实现了零周期延迟的数据转发。

#### d) 任务 B: MSHR 向外发送内存请求

```
230 | /*=====
231 | | 5. 任务B: MSHR向外发送内存请求
232 | |=====*/
233 | // 查找可以发出的请求: 有效、未发出、未完成
234 | val can_issue_vec = mshr_entries.zipWithIndex.map { case (entry, i) =>
235 | | entry.valid && !entry.issued && !entry.data_valid
236 | }
237 | val can_issue_oh = PriorityEncoderOH(can_issue_vec)
238 | val can_issue_id = OHToUInt(can_issue_oh)
239 | val can_issue_any = can_issue_vec.reduce(_ || _)
240 |
241 | // 发出内存请求
242 | io.mem_req_valid := can_issue_any
243 | io.mem_req_addr := Mux1H(can_issue_oh, mshr_entries.map(_.addr))
244 | io.mem_req_id := Mux1H(can_issue_oh, mshr_entries.map(_.req_id))
245 |
246 | // 更新issued状态
247 | when(can_issue_any && io.mem_req_ready) {
248 | | mshr_entries(can_issue_id).issued := true.B
249 | }
```

具体而言,它负责从已记录但尚未发出的缺失请求中,选择一个符合条件的请求发送给内存。

首先并行检查所有 MSHR 表项,筛选出那些“有效”、“尚未发出”且“未收到数据”的请求,形成一个布尔向量(`can_issue_vec`)。接着,通过优先级编码器(`PriorityEncoderOH`)从所有符合条件的请求中,找出索引值最小的那个(即优先级最高),并将其转换为独热码和对应的索引号。如果存在任何一个符合条件的请求(`can_issue_any` 为真),模块就会立即将相应的内存地址和请求 ID 通过多路选择器(`Mux1H`)输出到内存接口,并置起请求有效信号。

当内存端返回“准备好”信号(`mem_req_ready`)时,代码会更新被选中请求的表项状态,将其标记为“已发出”,从而确保同一个请求不会被重复发送。

### e) 任务 C：接收内存返回的数据

```
251 | /*=====
252 | | 6. 任务C: 接收内存返回的数据
253 | |=====*/
254 | // 根据ID查找对应的MSHR表项
255 | val rsp_match_vec = mshr_entries.zipWithIndex.map { case (entry, i) =>
256 | | entry.valid && entry.issued && (entry.req_id == io.mem_rsp_id)
257 | }
258 | val rsp_match_oh = PriorityEncoderOH(rsp_match_vec)
259 | rsp_match_id := OHToUInt(rsp_match_oh)
260 | rsp_match_any := rsp_match_vec.reduce(_ || _)
261 |
262 | // 更新MSHR表项的数据
263 | when(io.mem_rsp_valid && rsp_match_any) {
264 | | mshr_entries(rsp_match_id).data_valid := true.B
265 | | mshr_entries(rsp_match_id).data := io.mem_rsp_data
266 | }
267 |
268 | // 广播数据到Load Table并更新
269 | for (i <- 0 until LOAD_ENTRIES) {
270 | > | when(io.mem_rsp_valid && rsp_match_any && ...
286 | | }
287 | }
```

先通过并行比较内存返回包中的请求 ID（`io.mem_rsp_id`）与所有已发出的MSHR 表项的 ID，找出匹配的表项，并生成其索引（`rsp_match_id`）和匹配标志（`rsp_match_any`）。

当内存响应有效（`io.mem_rsp_valid`）且找到匹配项时，将完整的 512 位缓存行数据（`io.mem_rsp_data`）存入对应的 MSHR 表项，并标记其 `data_valid` 为真。

紧接着，广播所有 Load 表项，对于属于该匹配 MSHR 表项、且自身尚未收到数据的 Load 请求，根据其预先存储的块内偏移（`offset`）从 512 位数据中选择对应的 64 位数据，更新 Load 表项的数据字段并标记为有效，从而将内存返回的一次性大块数据精准地分发给所有相关的等待中的 CPU 读请求。

### f) 任务 D：写 Cache 操作

```
289 | /*=====
290 | | 7. 任务D: 写Cache操作 为缩减传播延迟，不依据内存的dataOK写cache，依据表项的written_cache信号写
291 | |=====*/
292 | // 查找可以写Cache的MSHR表项：数据有效但未写Cache
293 | val can_write_cache_vec = mshr_entries.zipWithIndex.map { case (entry, i) =>
294 | | entry.valid && entry.data_valid && !entry.written_cache
295 | }
296 | val can_write_cache_oh = PriorityEncoderOH(can_write_cache_vec)
297 | val can_write_cache_id = OHToUInt(can_write_cache_oh)
298 | val can_write_cache_any = can_write_cache_vec.reduce(_ || _)
299 |
300 | // 写Cache Data Tag接口
301 | io.cache_wr_valid := can_write_cache_any
302 | io.cache_wr_addr := Mux1H(can_write_cache_oh, mshr_entries.map(_.addr))
303 | io.cache_wr_data := Mux1H(can_write_cache_oh, mshr_entries.map(_.data))
304 |
305 | // 更新written_cache状态
306 | when(can_write_cache_any) {
307 | | mshr_entries(can_write_cache_id).written_cache := true.B
308 | }
```

此处负责在 MSHR 模块收到内存返回的完整缓存行数据后，安排将其写回



缓存。

它首先通过遍历 MSHR 表项，找出所有“有效、数据已就绪但尚未写入缓存”的条目，生成一个候选向量 `can_write_cache_vec`。接着，通过优先级编码器（`PriorityEncoderOH`）从这些候选条目中选出一个，将其索引转换为独热码和二进制 ID。

如果存在这样的候选条目（`can_write_cache_any` 为真），就会立即将对应的缓存行地址（`addr`）和数据（`data`）通过 `Mux1H` 选择器驱动到模块的输出接口（`io.cache_wr_*`），从而发起一次缓存写操作。

同时，在同一个周期内，它会将选中条目的 `written_cache` 状态位置为真，标记该缓存行“已写入”，以防止后续被重复调度写入。

这个设计将缓存更新决策与内存响应时序解耦，基于内部状态而非外部握手信号来触发写操作，有助于减少关键路径延迟。

#### g) 任务 E：返回数据给 CPU

```
310 | /*=====
311 | | 8. 任务E: 返回数据给CPU
312 | |=====*/
313 | // 查找可以返回给CPU的Load表项：数据有效但未返回
314 | val can_return_vec = load_entries.zipWithIndex.map { case (entry, i) =>
315 | | entry.valid && entry.data_valid && !entry.returned_cpu
316 | | }
317 | val can_return_oh = PriorityEncoderOH(can_return_vec)
318 | val can_return_id = OHToUInt(can_return_oh)
319 | val can_return_any = can_return_vec.reduce(_ || _)
320 |
321 | // CPU返回接口
322 | io.cpu_rsp_valid := can_return_any
323 | io.cpu_rsp_data := Mux1H(can_return_oh, load_entries.map(_.data))
324 | io.cpu_rsp_id := Mux1H(can_return_oh, load_entries.map(_.req_id))
325 | io.cpu_rsp_idx := Mux1H(can_return_oh, load_entries.map(_.req_idx))
326 |
327 | // 更新returned_cpu状态
328 | when(can_return_any && io.cpu_rsp_ready) { //要外界能够返才返，避免与外界HIT返数据冲突的情况发生
329 | | load_entries(can_return_id).returned_cpu := true.B
330 | }
```

首先通过遍历 Load 表项数组，并行检查哪些表项满足三个条件：有效、数据已准备就绪但尚未返回给 CPU。根据检查结果，它生成一个布尔向量（`can_return_vec`），并使用优先级编码器（`PriorityEncoderOH`）找到第一个满足条件的表项，生成其独热码和索引。

如果存在任何这样的表项，则置位 CPU 响应有效信号（`io.cpu_rsp_valid`），并利用独热码通过多路选择器（`Mux1H`）从对应的 Load 表项中选取数据、请求 ID 和寄存器索引，驱动到 CPU 接口。

最后，代码在 CPU 接口确认就绪（io.cpu\_rsp\_ready）的同一个周期，将对应 Load 表项的 returned\_cpu 状态置为真，标记该请求的数据已成功交付，从而完成了从内存响应到 CPU 响应的最终数据通路。

## h) 清理已完成表项

```

332 | /*=====
333 | | 9. 清理已完成表项
334 | |=====*/
335 | // 清理MSHR表项: 数据已写Cache, 且关联的所有Load请求都已完成
336 | for (i <- 0 until MSHR_ENTRIES) {
337 |   val mshr_entry = mshr_entries(i)
338 |
339 |   when(mshr_entry.valid && mshr_entry.written_cache && !write_mshr) { //&& !write_mshr: 后面发
340 |     }
341 | }
342 |
343 | //((entry.data_valid && entry.returned_cpu)
344 | // 清理Load表项: 已返回CPU
345 | for (i <- 0 until LOAD_ENTRIES) {
346 |   when(load_entries(i).valid && load_entries(i).returned_cpu && !write_load) { ...
347 |     }
348 | }

```

对于 MSHR 表项，当一个条目有效（valid）、其数据已成功写入缓存（written\_cache）、且当前没有新的写请求（!write\_mshr，此条件用于防止清理操作与写操作发生冲突）时，该表项的所有状态位和存储数据（包括地址、请求 ID 和缓存行数据）将被清零，使其恢复为空闲状态。

对于 Load 表项，当一个条目有效、其数据已成功返回给 CPU（returned\_cpu）、且当前没有新的 Load 请求写入（!write\_load）时，该表项的所有字段（包括关联的 MSHR ID、请求 ID、数据、偏移量等）也会被清零。这两个清理过程通过循环遍历所有表项并在条件满足时并行执行，确保了模块能够持续复用其有限数量的表项来处理后续的缓存未命中请求。

## 4. 验证框架设计

### 4.1. 功能测试总体思路

请参见代码 `src/test/CacheSpec.scala`

验证环境主要利用了 ChiselTest（基于 ScalaTest）工具。

```
10 it should "using random delays through randomly generated test cases" in {
11
12   test(new NonBlockingCache(6,36,6))
13     .withAnnotations(Seq(WriteVcdAnnotation)) { dut =>
14
15     dut.clock.setTimeout(0)
16
17     val ram = new FakeNonBlockingRAM(dut)
18     val cpu = new FakeCPU(dut, TEST_NUM)
```

具体而言，该测试用例构建了一个完整的仿真环境来验证非阻塞缓存的设计。它首先实例化被测的缓存设计模块（DUT），并创建模拟下层内存的 FakeNonBlockingRAM 和模拟 CPU 请求的 FakeCPU 对象。

```
20   dut.reset.poke(true.B)
21   dut.clock.step(10)
22   dut.reset.poke(false.B)
23   var stopEarly = false
24
25   for (cycle <- 0 until TEST_CYCEL if !stopEarly) {
26     //println(s"---- cycle $cycle ----")
27     ram.step(cycle)
28     cpu.step(cycle)
29     dut.clock.step(1)
30     if(tested_num == TEST_NUM){
31       stopEarly = true //全部测试结束后立即停止
32     }
33   }
34
35   println(f"Tested Num: $tested_num%-4d All Test Num: $TEST_NUM%-4d ")
```

在完成复位操作后，测试进入一个主循环，在每个仿真周期内，它依次驱动内存模拟器、CPU 模拟器，并推进被测设计的时钟。整个测试会持续执行预设的周期数（TEST\_CYCEL），但如果 CPU 模拟器报告其已完成所有预定的测试请求（tested\_num == TEST\_NUM），测试会提前终止。

最终，代码会打印出实际执行的测试请求数量与总请求数量的对比信息，从而验证缓存功能在随机内存延迟下的正确性。

## 4.2. FakeCPU 的设计思路

请参见代码 `src/test/FakeCPU.scala`

FakeCPU 模块通过模拟 CPU 的请求行为来测试非阻塞 Cache 的正确性：

随机生成请求的测试数据：

```
37 // 请求地址生成函数
38 def randomAddrWithLocality(hotBases: Vector[BigInt], localityRatio: Double = 0.8): BigInt = {
39     val offsetBits = 6
40     val indexBits = 6
41     val tagBits = 36
42     val rand = rng.nextDouble()
43     if (rand < localityRatio * 0.7 && hotBases.nonEmpty) {
44         // 80% * 70% 的局部性: 相同tag+index, 不同offset
45         val base = hotBases(rng.nextInt(hotBases.size))
46         val offset = BigInt(rng.nextInt(1 << offsetBits))
47         base | offset
48     } else if (rand < localityRatio && conflictIndices.nonEmpty) {
49         // 80% * 30% 的局部性: 使用相同index, 但生成新的tag
50         val baseIndexOnly = conflictIndices(rng.nextInt(conflictIndices.size))
51         // 提取index部分
52         val index = (baseIndexOnly >> 6) & ((1 << indexBits) - 1)
53         val tag = BigInt(tagBits, rng) | (BigInt(1) << (tagBits - 1))
54         val offset = BigInt(rng.nextInt(1 << offsetBits))
55         (tag << (indexBits + offsetBits)) | (index << offsetBits) | offset
56     } else {
57         // 20% 完全随机生成地址
58         val tag = BigInt(tagBits, rng) | (BigInt(1) << (tagBits - 1))
59         val index = rng.nextInt(1 << indexBits)
60         val offset = BigInt(rng.nextInt(1 << offsetBits))
61         (tag << (indexBits + offsetBits)) | (BigInt(index) << offsetBits) | offset
62     }
63 }
```

基于固定随机种子生成一组带有局部性特征的测试序列，其中 80%的地址访问具有局部性（70%访问同一缓存行的不同偏移，30%访问同一索引的不同标签），20%为完全随机地址。

```
TestData > Req.txt
1  ===== CPU INIT REQUESTS =====
2  This data is generated with a fixed random seed during simulation testing.
3  The request data will contain a certain proportion of test hits/replacement cache samples, rather
4  For detailed code, see: src/test/scala/FakeCPU.scala
5  REQ00 : addr=0x8a505179bda1 rd=21
6  REQ01 : addr=0x9d83f5c229d5 rd=18
7  =====
8  |
```

生成数据会存储到 `./TestData/Req.txt` 文件中便于调试。

以随机延迟发送请求数据：

```

104     if (sending && sendPtr < requests.length) {
105         val req = requests(sendPtr)
106         dut.io.req.poke(true.B)
107         dut.io.addr.poke(req.addr.U)
108         dut.io.rd_idx_in.poke(req.rdIdx.U)
109         dut.io.sendReq_in.poke(sendPtr.U)
110
111         if (dut.io.addr_ok.peekBoolean()) {
112             //println(s"HI3")
113
114             println(f" ")
115             println(
116                 f"[CPU ] SEND      [Time ${dut.io.counter.peekInt()}%4d] REQ [{sendPtr}%3d] addr=0x${req.addr.toString(16)}
117             )
118             sendPtr += 1
119             if (sendPtr < requests.length){
120                 sending = rng.nextBoolean() //决定下一个周期还发不发
121             }
122             else{
123                 sending = false
124             }
125         }
126     } else {
127         if (sendPtr < requests.length) {
128             sending = rng.nextBoolean()
129         }
130     }
131 }

```

在每个测试周期中，FakeCPU 以随机间隔向 Cache 发送请求，当收到 Cache 的地址确认时记录请求发送。

### 接收响应数据并进行验证：

```

133     if (dut.io.data_ok.peekBoolean()) {
134         val data = dut.io.rdata.peekInt()
135         val rdIdx = dut.io.rd_idx_out.peekInt()
136         val sendReq = dut.io.sendReq_out.peekInt()
137         println(f" ")
138         println(
139             f"[CPU ] RECIEVE [Time ${dut.io.counter.peekInt()}%4d] REQ [{sendReq}%3d] rd=${rdIdx}%2d data=0x${data}
140         )
141         val req_verify = requests(sendReq.toInt)
142         val addr_verify = req_verify.addr
143         //println(s" Addr_verify -> 0x${format64Hex(addr_verify)}")
144         val rdIdx_verify = req_verify.rdIdx
145         val shifted = addr_verify >> 6
146         val lineAddr = {
147             val mod = shifted % RAM_LINES
148             if (mod >= 0) mod.toInt
149             else (mod + RAM_LINES).toInt
150         }
151         val data_verify = memory(lineAddr.toInt)
152         val bankIndex = ((addr_verify >> 3) & 0x7).toInt // 提取第3-5位 (bank选择位)
153         // 从512位数据中提取对应的64位bank
154         val selectedDataVerify = extractBankFrom512(data_verify, bankIndex)
155         println(f" |")
156         println(f" | [VERIFY] SendReq: $sendReq%-4d |")
157         println(f" |")
158         println(f" | Addr      : 0x${format64Hex(addr_verify)} |")
159         println(f" | LineIndex : 0x${format8Hex(lineAddr)} |")
160         println(f" | BankIndex : 0x${format8Hex(bankIndex)} |")

```

当收到数据响应时，则从预先生成的内存数据中提取对应地址的期望值，与实际返回数据进行比对验证。验证效果如下：

```
if(data == selectedDataVerify && rdIdx == rdIdx_verify){
    println(s"      -----> PASS")
}else{
    println(s"      -----> ERROR!!!")
    sys.error(s"VERIFICATION FAILED!")
}

tested_num = tested_num + 1
```

[CPU ] RECIEVE [Time 32] REQ [ 0] rd=21 data=0x3bc67b4522d425d6

|                     |                                      |
|---------------------|--------------------------------------|
| [VERIFY] SendReq: 0 |                                      |
| Addr                | : 0x00008a505179bda1                 |
| LineIndex           | : 0x36                               |
| BankIndex           | : 0x04                               |
| Expected            | : Data: 0x3bc67b4522d425d6 RdIdx: 21 |
| Actual              | : Data: 0x3bc67b4522d425d6 RdIdx: 21 |

```
-----> PASS
Tested Num: 2      All Test Num: 2
[info] CacheSpec:
[info] Non-BlockingCache
[info] - should using random delays through randomly generated test cases
[info] Run completed in 19 seconds, 787 milliseconds.
[info] Total number of tests run: 1
[info] Suites: completed 1, aborted 0
[info] Tests: succeeded 1, failed 0, canceled 0, ignored 0, pending 0
[info] All tests passed.
[success] Total time: 24 s, completed Feb 20, 2026, 12:33:03 PM
```

### 4.3. FakeRAM 的设计思路

请参见代码 `src/test/ FakeNonBlockingRAM.scala`

FakeNonBlockingRAM 模块通过模拟非阻塞、带随机延迟的内存行为来验证 Cache 的正确性。在测试程序开始时会随机生成 Memory 数组模拟内存数据，详细结果数据存于：TestData/Ram.txt

```

TestData > Ram.txt
1 | Generated with a random seed during test program execution
2 | For detailed code, see: src/test/scala/Tool.scala
3 |

```

| SYSTEM RAM CONTENTS (512-bit) |                                                                                                                                                              |
|-------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Addr                          | Data                                                                                                                                                         |
| 0x00                          | 984bc005fc0d8e66<br>5bb2068bbe3e36d8<br>8858bde377b2b60c<br>f274dd02370d0be7<br>4c29a3ee0ff80126<br>d43377b8719268c8<br>8e7835e558f2152a<br>fd9e5356ed115020 |
| 0x01                          | 70cf1c91bdda05e6<br>69b33380c6d179d2<br>aa4ee491d90c4792                                                                                                     |

接收并管理来自 Cache 的内存访问请求：

```

84 | /*=====
85 | 1. FakeRAM接收来自Cache的请求
86 | =====*/
87 |
88 | if (dut.io.mem_req.peekBoolean()) {
89 |     // random addr accept
90 |     if (rng.nextBoolean()) {
91 |         val addr = dut.io.mem_addr.peekInt()
92 |         val rdaddr = 0//dut.io.mem_rdaddr.peekInt()
93 |         val sendReq = dut.io.mem_sendReq_out.peekInt()
94 |
95 |         dut.io.mem_addr_ok.poke(true.B)
96 |         val ready = cycle + randDelay(5, 20)
97 |
98 |         pending += PendingReq(addr, rdaddr, sendReq, ready)
99 |         println(f"[RAM ] RECEIVE [Time ${dut.io.counter.peekInt()}%4d] REQ [${sendReq}%3d] addr=0x${format64He
.00 |     }
.01 | }
.02 |

```

首先，模块通过一个待处理请求队列（pending）接收并管理来自 Cache 的内存访问请求，每个请求会被分配一个在未来 5-20 个随机周期后“就绪”的时间戳，模拟了内存访问延迟的不确定性。

读取 RAM 数据，并启动突发传输：



```

103 | /*=====
104 | 2. FakeRAM向Cache突发传输对应的数据
105 | =====*/
106 | if (burstState.isEmpty) {
107 |     val readyList = pending.filter(_.readyCycle <= cycle)
108 |     if (readyList.nonEmpty) {
109 |         val idx = rng.nextInt(readyList.length)
110 |         val req = readyList[idx]
111 |
112 |         val shifted = req.addr >> 6
113 |         val lineAddr = {
114 |             val mod = shifted % RAM_LINES
115 |             if (mod >= 0) mod.toInt
116 |             else (mod + RAM_LINES).toInt
117 |         }
118 |         val data = memory(lineAddr.toInt)
119 |         //println(s" [RAM] Addr -> 0x${format64Hex(req.addr)}")
120 |         //println(s" [RAM] lineAddr -> 0x${format64Hex(lineAddr)}")
121 |         //println(s" [RAM] CacheLineData -> 0x${format512Hex(data)}")
122 |         // 开始新的突发传输
123 |         burstState = Some(BurstState(...
124 |     ))
125 |
126 |     println(
127 |         f"[RAM ] BACKDATA [Time ${dut.io.counter.peekInt()}%4d] REQ [${req.sendReq}%3d] addr=0x${format64Hex(req.ar
128 |     )
129 |     )
130 |     println(
131 |         f"                                     data=0x${format512Hex(data)}"

```

当请求就绪时，模块会根据请求地址计算对应的内存行索引，从预先生成的全局内存数组（memory）中读取完整的 512 位数据，并启动一个 8 周期的突发传输状态（burstState）。

```

54 | burstState match {
55 |     case Some(state) =>
56 |         // 突发传输进行中
57 |         val bankIdx = state.burstCount
58 |         if (bankIdx < BURST_SIZE) {
59 |
60 |             val bankData = extractBankFrom512(state.data, bankIdx)
61 |             // 设置输出信号
62 |             dut.io.mem_data_valid.poke(true.B)
63 |             dut.io.mem_rdata.poke(bankData.U)
64 |             //dut.io.mem_rdata_512.poke(state.data.U)
65 |             //dut.io.mem_addr_back.poke(state.addr.U)
66 |             //dut.io.mem_rdaddr_back.poke(state.rdaddr.U)
67 |             dut.io.mem_sendReq_back.poke(state.sendReq.U)
68 |
69 |             // 如果是最后一个数据，设置data_ok
70 |             if (bankIdx == BURST_SIZE - 1) {
71 |                 dut.io.mem_data_ok.poke(true.B)
72 |             }
73 |             // 更新burst状态
74 |             burstState = Some(state.copy(burstCount = bankIdx + 1))
75 |         } else {
76 |             // 突发传输完成，清除状态
77 |             burstState = None
78 |         }

```

在突发传输过程中，模块每个周期向 Cache 接口发送一个 64 位的数据片段（mem\_data\_valid 为高），并在传输最后一个数据时置高完成信号（mem\_data\_ok），从而模拟了真实内存的突发传输时序。该设计确保了 Cache 模块能在乱序响应和随机延迟的环境下得到充分测试。



## 5. 仿真与验证

### 5.1. 仿真验证参数

本文档详细呈现了在随机种子为 6688、测试请求数量为 1000 条时的完整仿真与验证结果。为充分验证设计的健壮性，亦在其他多种随机种子下进行了测试，均通过功能验证。

本部分仅以种子 6688 为例展开详细说明。该次验证的完整波形文件(VCD)与结果截图已存档于项目 **doc/验证结果** 目录下。如需调整测试规模或更换测试种子，可在 `src/test/scala/GlobalVars.scala` 中修改对应参数，重新执行测试即可。

```
2  object GlobalVars {  
3  
4      val TEST_NUM = 1000 /*修改此处需要测试的样例数量*/ //全部测试结束后自动停止  
5      var randomSeed: Long = 6688L /*修改此处的随机种子以生成 不同测试测样 及 不同随机延迟*/  
6  
7  
8      val TEST_CYCEL = 10000 //最多测试的周期数  
9      var tested_num: Int = 0 //记录已测试的数量  
10 }
```

### 5.2. 验证结果

在随机种子为 6688、测试请求数量为 1000 条的测试配置下，基于【4.验证框架设计】所构建的验证环境，该 Non-Blocking Cache 成功通过了全部功能测试。具体测试结果如下：

```

[CPU ] SEND      [Time 4567] REQ [998] addr=0x9f997f7fef75 rd=18
[CPU ] RECIEVE   [Time 4568] REQ [998] rd=18 data=0xd8ff38089e39d62c

```

|                       |                                      |
|-----------------------|--------------------------------------|
| [VERIFY] SendReq: 998 |                                      |
| Addr                  | : 0x00009f997f7fef75                 |
| LineIndex             | : 0x3d                               |
| BankIndex             | : 0x06                               |
| Expected              | : Data: 0xd8ff38089e39d62c RdIdx: 18 |
| Actual                | : Data: 0xd8ff38089e39d62c RdIdx: 18 |

```

-----> PASS
[CPU ] SEND      [Time 4572] REQ [999] addr=0xe5b087a43112 rd= 8
[CPU ] RECIEVE   [Time 4573] REQ [999] rd= 8 data=0xa22d637d9976b1a7

```

|                       |                                     |
|-----------------------|-------------------------------------|
| [VERIFY] SendReq: 999 |                                     |
| Addr                  | : 0x0000e5b087a43112                |
| LineIndex             | : 0x04                              |
| BankIndex             | : 0x02                              |
| Expected              | : Data: 0xa22d637d9976b1a7 RdIdx: 8 |
| Actual                | : Data: 0xa22d637d9976b1a7 RdIdx: 8 |

```

-----> PASS
[CPU ] RECIEVE   [Time 4575] REQ [990] rd=25 data=0xae50a705e4d4c261

```

|                       |                                      |
|-----------------------|--------------------------------------|
| [VERIFY] SendReq: 990 |                                      |
| Addr                  | : 0x00008115fa11f652                 |
| LineIndex             | : 0x19                               |
| BankIndex             | : 0x02                               |
| Expected              | : Data: 0xae50a705e4d4c261 RdIdx: 25 |
| Actual                | : Data: 0xae50a705e4d4c261 RdIdx: 25 |

```

-----> PASS
Tested Num: 1000 All Test Num: 1000
[info] CacheSpec:
[info] Non-BlockingCache
[info] - should using random delays through randomly generated test cases
[info] Run completed in 23 minutes, 3 seconds.
[info] Total number of tests run: 1
[info] Suites: completed 1, aborted 0
[info] Tests: succeeded 1, failed 0, canceled 0, ignored 0, pending 0
[info] All tests passed.
[success] Total time: 1393 s (0:23:13.0), completed Feb 20, 2026, 12:07:53 PM

```

### 5.3.测试结果分析

本节通过波形图追踪并剖析一条缓存未命中请求与一条缓存命中请求在Cache内部的完整生命周期，旨在从时序细节层面验证缓存各模块协同工作的功能正确性。

### 5.3.1 对于一条 MISS 的请求

此次选择 REQ0，即首个 CPU 请求作为分析对象：

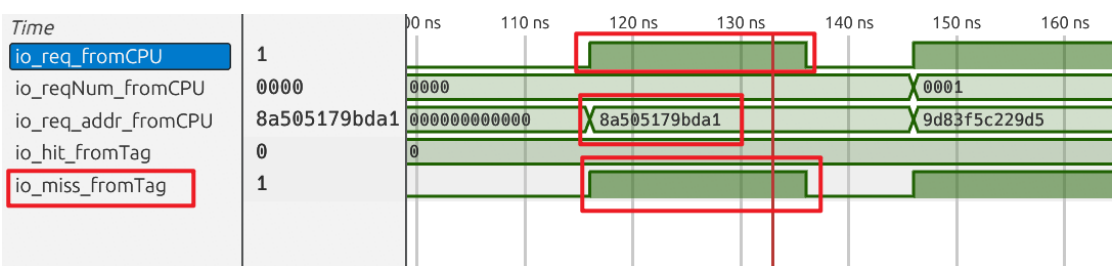
```
TestData > Req.txt
1  ===== CPU INIT REQUESTS =====
2  This data is generated with a fixed random seed during simulation testing.
3  The request data will contain a certain proportion of test hits/replacement cache samples, rather than being compl
4  For detailed code, see: src/test/scala/FakeCPU.scala
5  REQ00 : addr=0x8a505179bda1 rd=21
6  REQ01 : addr=0x9d83f5c229d5 rd=18
7  REQ02 : addr=0xef664a778da7 rd=11
8  REQ03 : addr=0x8186e5d77daa rd=27
9  REQ04 : addr=0x9f997f7fef7c rd=5
10 REQ05 : addr=0x93e560e23455 rd=20
11 REQ06 : addr=0xc8c76e6ffffef rd=3
12 REQ07 : addr=0xeea7b19b1919 rd=19
13 REQ08 : addr=0xcf39bb21ad88 rd=28
14 REQ09 : addr=0xb2daa0269445 rd=19
```

该请求将内存地址 0x8a505179bda1 处的数据加载至第 21 号寄存器。

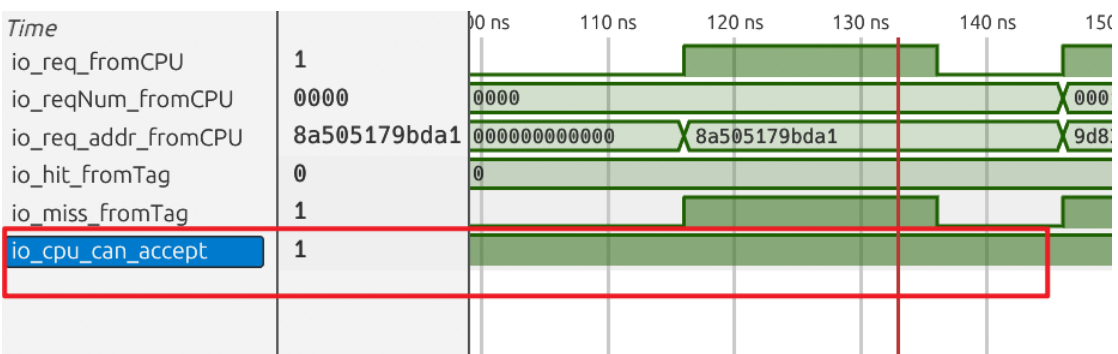
```
[info] set current project to nonblockingcache (in build file:/mnt/d/myCPU_new/NonBlockingCache/)
[info] compiling 3 Scala sources to /mnt/d/myCPU_new/NonBlockingCache/target/scala-2.13/classes ...
[info] compiling 5 Scala sources to /mnt/d/myCPU_new/NonBlockingCache/target/scala-2.13/test-classes ...
Generating RAM data in ./TestData/Ram.txt .....
Generating request data in ./TestData/Req.txt .....
Start testing .....

[CPU ] SEND      [Time  4] REQ [ 0] addr=0x8a505179bda1 rd=21
```

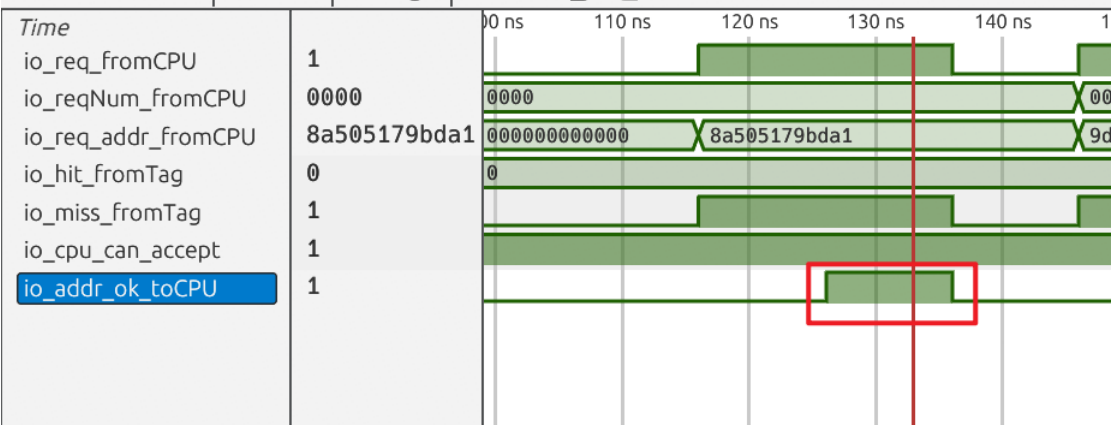
在仿真时间 4（cycle 4）时，FakeCPU 发出该请求，其时序波形如图所示：



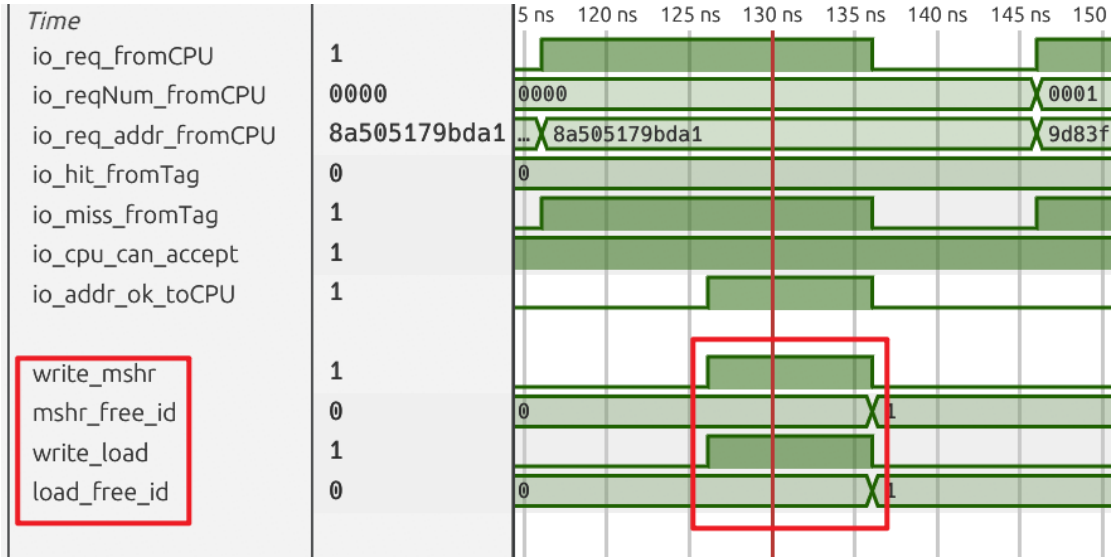
根据 Tag 的检测结果，该请求处于 Miss 状态。接下来检查 MSHR 是否能接收该请求：



可以接收，并将 **addrOK** 信号返还给 CPU：



行为正确，同时在该周期内也将对 **MSHR** 表与加载表进行写入：



如图可见，两张表均已完成写入操作，且均指向第 **0** 个表项，可进一步核查该表项的实际写入状态：



```

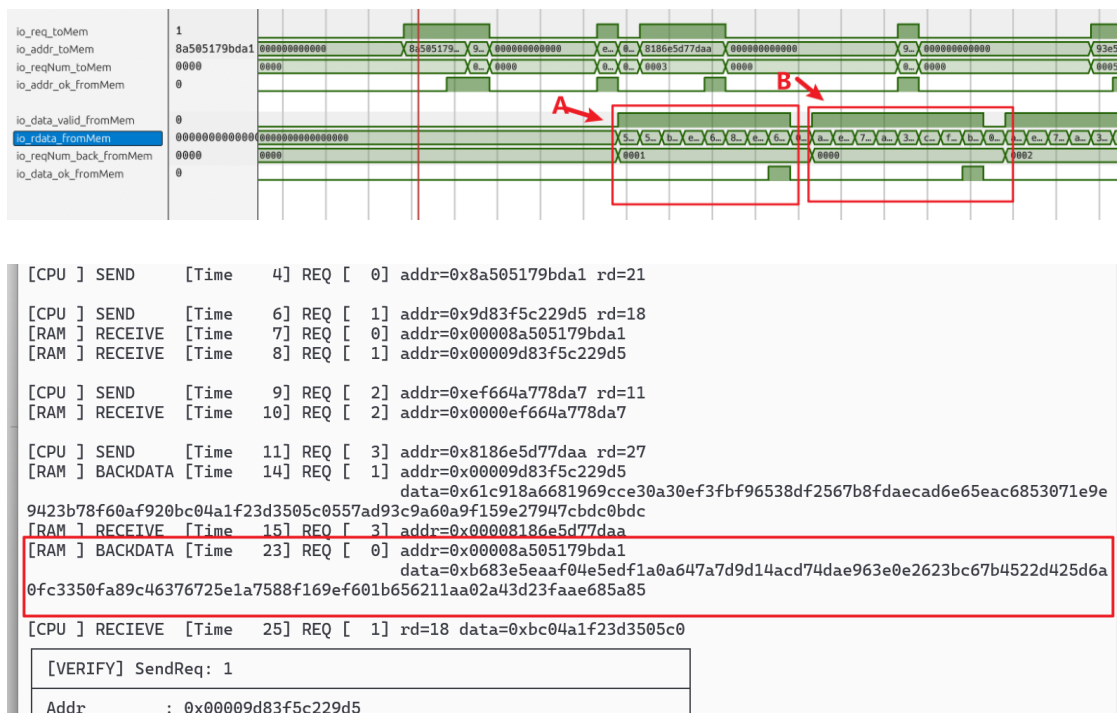
[CPU ] SEND      [Time   4] REQ [ 0] addr=0x8a505179bda1 rd=21
[CPU ] SEND      [Time   6] REQ [ 1] addr=0x9d83f5c229d5 rd=18
[RAM ] RECEIVE   [Time   7] REQ [ 0] addr=0x00008a505179bda1
[RAM ] RECEIVE   [Time   8] REQ [ 1] addr=0x00009d83f5c229d5

[CPU ] SEND      [Time   9] REQ [ 2] addr=0xef664a778da7 rd=11
[RAM ] RECEIVE   [Time  10] REQ [ 2] addr=0x0000ef664a778da7

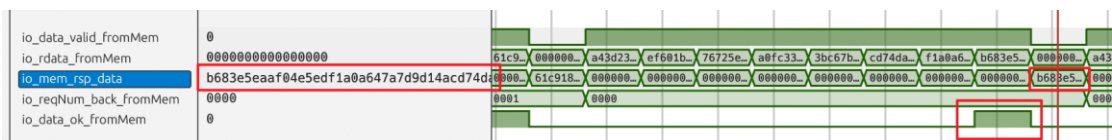
```

可观察到，MSHR 立即发出请求，并在之后几个周期被 RAM 接收。

随后继续观察 RAM 在何时返回数据：

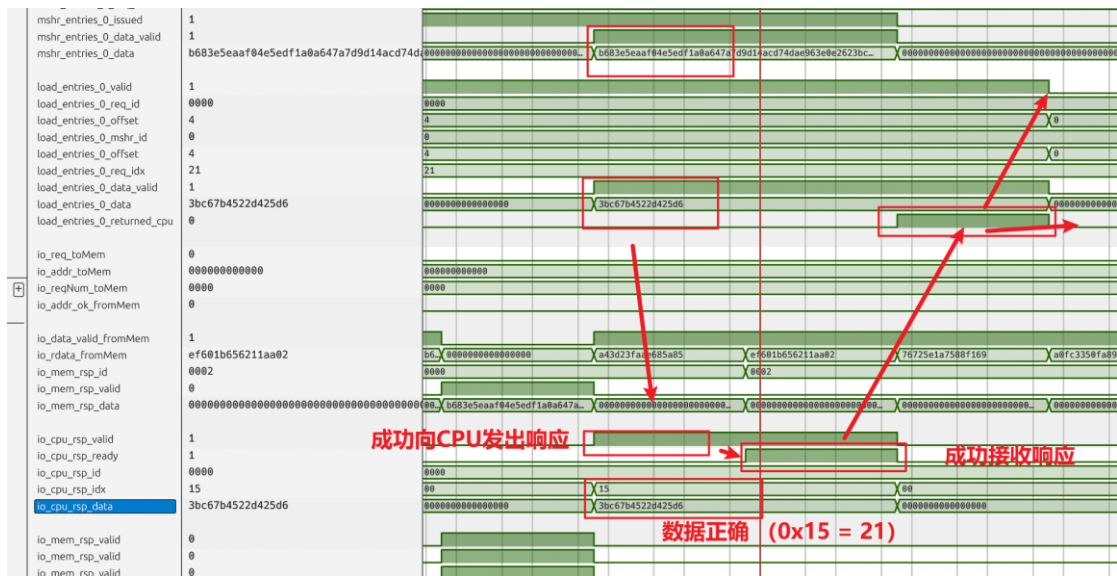


可见，在若干周期后，RAM 返回了数据：A 时刻返回的是 ID 为 1 的请求数据，B 时刻返回的是 ID 为 0 的请求数据。因此，我们需要重点分析 B 时刻返回的数据。



成功将接收到的突发传输数据组合进缓存行。





该结果在被填入响应表项后即可返回给 CPU，并在返回后自动清零，可见其行为均符合预期。

|                                                                 |                                      |
|-----------------------------------------------------------------|--------------------------------------|
| [CPU ] RECIEVE [Time 34] REQ [ 0] rd=21 data=0x3bc67b4522d425d6 |                                      |
| [VERIFY] SendReq: 0                                             |                                      |
| Addr                                                            | : 0x00008a505179bda1                 |
| LineIndex                                                       | : 0x36                               |
| BankIndex                                                       | : 0x04                               |
| Expected                                                        | : Data: 0x3bc67b4522d425d6 RdIdx: 21 |
| Actual                                                          | : Data: 0x3bc67b4522d425d6 RdIdx: 21 |
| -----> PASS                                                     |                                      |

验证也通过。

### 5.3.2 对于一条 HIT 的请求

此次选择 REQ15，第 15 条请求：

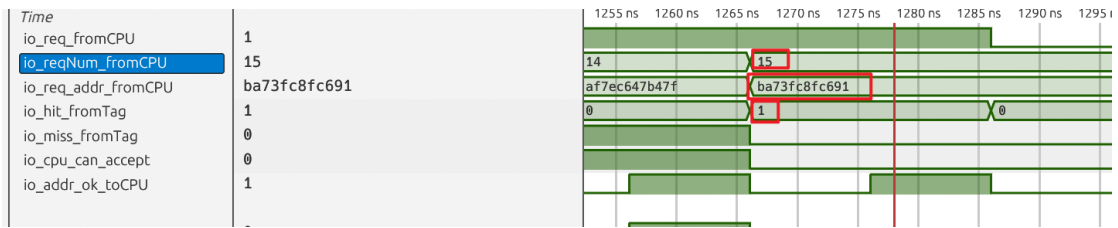
```

12 REQ07 : addr=0xeea7b19b1919 rd=19
13 REQ08 : addr=0xcf39bb21ad88 rd=28
14 REQ09 : addr=0xb2daa0269445 rd=19
15 REQ10 : addr=0xb713461a0c5e rd=2
16 REQ11 : addr=0xba73fc8fc6a6 rd=18
17 REQ12 : addr=0xcbddf07c0c55 rd=11
18 REQ13 : addr=0xb40c93b1d797 rd=12
19 REQ14 : addr=0xaf7ec647b47f rd=27
20 REQ15 : addr=0xba73fc8fc691 rd=4
21 REQ16 : addr=0xcf39bb21ad94 rd=4
22 REQ17 : addr=0xbbd5dbe53993 rd=17
23 REQ18 : addr=0xf4682d150d91 rd=29

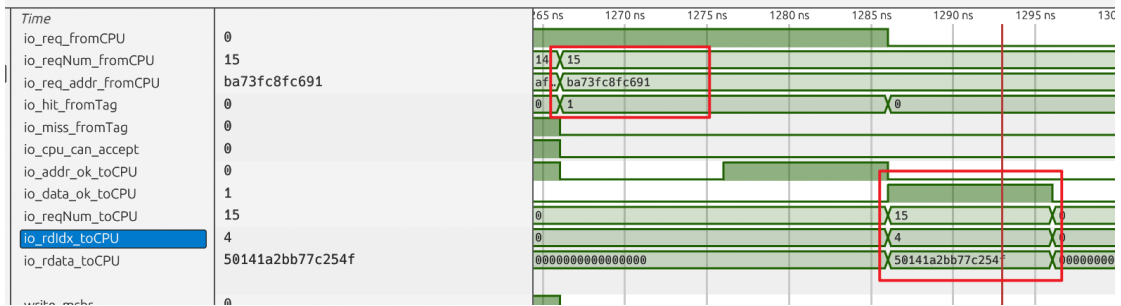
```

该指令将从内存地址 0xba73fc8fc691 读取数据，并加载至第四号寄存器中。





由此可见，此时 Cache 命中，随即返回了 addrOK。



行为正确，并在下一周期成功返回 dataOK 及读取的数据。。

|   |     |      |                  |
|---|-----|------|------------------|
| M | 240 |      | f7a01acaad1403f4 |
| M | 241 |      |                  |
| M | 242 | 0x1a | 2e8921af03af3e0a |
| M | 243 |      | a4eb295fb57a8b93 |
| M | 244 |      | b63591d806bdf487 |
| M | 245 |      | 8ad8359265a1a2fa |
| M | 246 |      | e02c38bc730fa9e6 |
|   | 247 |      | 50141a2bb77c254f |
|   | 248 |      | 916480b2900d7748 |
|   | 249 |      | bedd94552f98393f |
|   | 250 |      |                  |
|   | 251 | 0x1b | c1da0e8d9b8de376 |
|   | 252 |      | 3ba343f20b01020  |

数据显示匹配，缓存行为完全符合预期。

[CPU ] RECIEVE [Time 120] REQ [ 15] rd= 4 data=0x50141a2bb77c254f

|                      |                                     |
|----------------------|-------------------------------------|
| [VERIFY] SendReq: 15 |                                     |
| Addr                 | : 0x0000ba73fc8fc691                |
| LineIndex            | : 0x1a                              |
| BankIndex            | : 0x02                              |
| Expected             | : Data: 0x50141a2bb77c254f RdIdx: 4 |
| Actual               | : Data: 0x50141a2bb77c254f RdIdx: 4 |

-----> PASS

自动验证同样可行。